РЕКЛАМА
Хабр Курсы
Взлетайте в профессии и в зарплате
На витрину

TheSeCure 20 фев 2025 в 17:51

Большой обзор больших языковых моделей

9 мин 44K

Блог компании Газинформсервис, Искусственный интеллект, Natural Language Processing*, Машинное обучение*

Обзор

Большая языковая модель — что это такое?

LLM (Large Language Model, большая языковая модель) — это тип модели машинного обучения (или тип модели глубокого обучения), основанный на больших объёмах данных, который может выполнять любые задачи по обработке естественного языка (Natural Language Processing, NLP).

Более простыми словами это нейронная сеть с крайне большим количеством изменяемых параметров, которая позволяет решать задачи по обработке и генерации текста. Чаще всего реализована в виде диалогового агента, с которым можно общаться в разговорной форме.

NLP — область информатики, который включает в себя предоставление компьютерам возможности интерпретировать и воспроизводить язык. В то же время NLP охватывает несколько областей обучения, включая лингвистику, информатику, статистику и искусственный интеллект. Основные сложности NLP заключаются в генерации и интерпретации человеческой речи, так как она достаточно двусмысленна и иногда даже люди, зная контекст, с трудом понимают, что имеется в виду. Например:

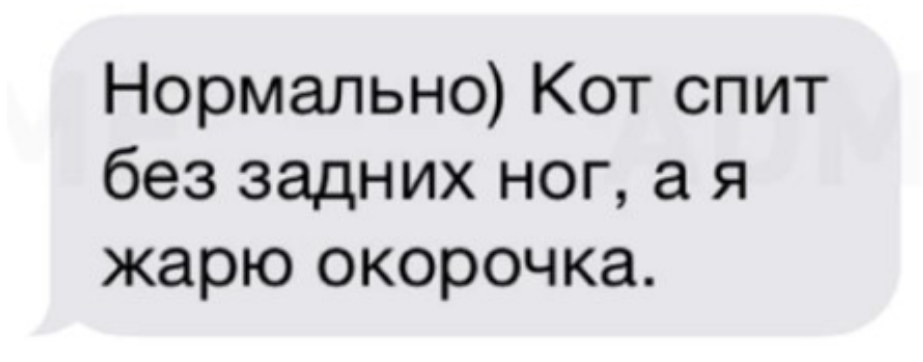

Нормально) Кот спит без задних ног, а я жарю окорочка.

Рис. 1 Двусмысленная фраза

Вот контекст:

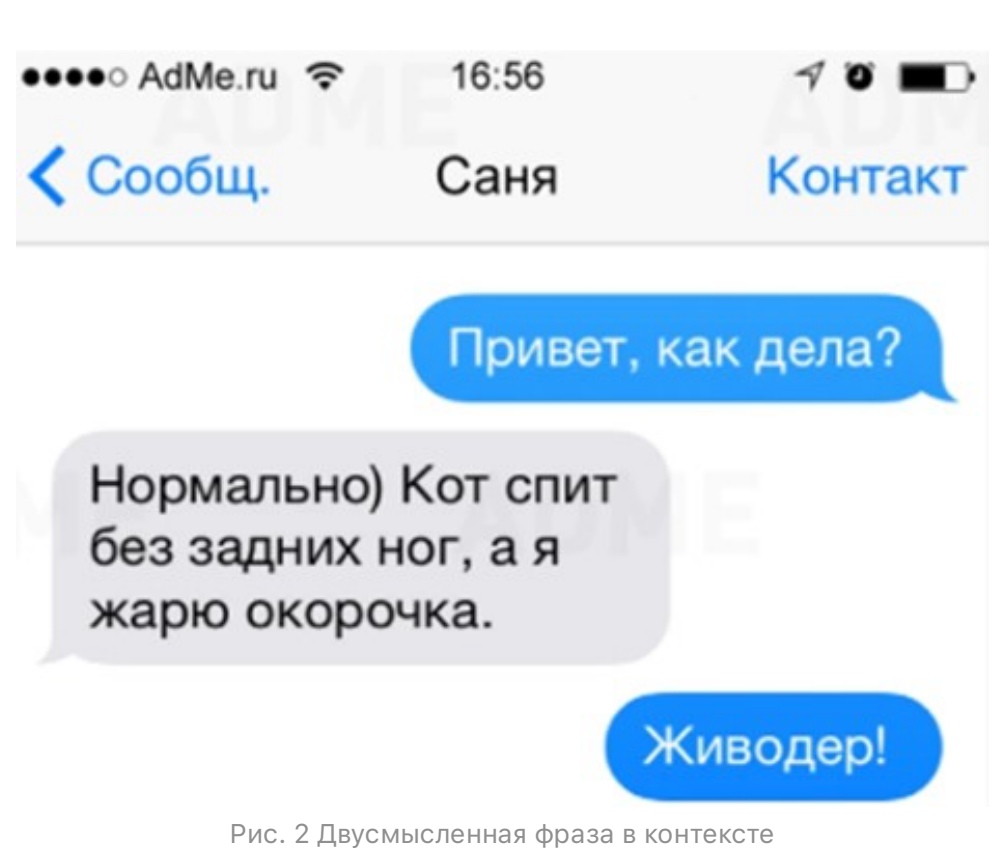

AdMe.ru
16:56
Сообщ.
Саня
Контакт
Привет, как дела?
Нормально) Кот спит без задних ног, а я жарю окорочка.
Живодер!

Рис. 2 Двусмысленная фраза в контексте

В самом начале хочется выделить ряд источников, которые подробно и понятно объясняют LLM:

- Для тех, кто не знаком с ML;
- База для NLP;
- Хорошая обзорная статья;
- Здесь качественно рассказаны основные подходы к разработке LLM.

История: от Тьюринга до GPT4

Существует много дискуссий о том, кто является создателем искусственного интеллекта. В этом исследовании отдаётся предпочтение основоположнику современных компьютеров — Алану Тьюрингу. Будем считать отправной точкой **1950 год**, когда он опубликовал статью, в которой ввёл понятие *«Тест Тьюринга»* для оценки способности машины имитировать человеческое поведение.

Следующим ярким событием была Дартмутская конференция в **1956 году**, на которой исследователи Джон Маккарти, Марвин Минский, Натаниэль Рочестер и Клод Шеннон предложили термин «искусственный интеллект» и сформулировали основные задачи и направления исследований в этой области. Большинство исследователей согласны с этой датой и считают конференцию началом эпохи искусственного интеллекта.

Рассматривая хронологию становления и развития LLM и ИИ в целом, можно выделить следующий ряд ключевых событий:

1967 год — появляется проект «Элиза», одна из первых программ, способная имитировать беседу с человеком.

1997 год — IBM Deep Blue побеждает чемпиона мира по шахматам Гарри Каспарова.

2006 год — Хинтон и Салахутдинов создают алгоритм глубокого обучения.

2011 год — IBM Watson побеждает в телешоу 'Jeopardy!', демонстрируя продвинутые возможности в области обработки естественного языка.

2017 год — Nvidia создала GPU для глубокого обучения.

2017 год — представлена архитектура Transformer.

2018 год — опубликована модель BERT, разработанная Google, которая использует архитектуру трансформеров и механизм внимания для глубокого понимания текста. Модель BERT представлена в двух основных версиях с различным количеством параметров:

- BERT Base содержит 110 миллионов параметров.
- BERT Large обладает 340 миллионами параметров.

2020 год — выпущена модель GPT-3 (Generative Pre-trained Transformer 3), разработанная OpenAI. Модель основана на архитектуре трансформеров, но использует однонаправленный (авторегрессионный) подход для генерации текста и обладает 175 миллиардами параметров. GPT-1 и GPT-2 обладают меньшим числом параметров: 117 миллионов у GPT-1 и 1,5 миллиарда у GPT-2 соответственно.

2021 год — DeepMind анонсирует AlphaFold для предсказания структуры белков.

2023 год — выпущена модель GPT-4.

2024 год — развиваются технологии генерации видео с использованием ИИ.

На рисунке ниже представлена краткая эволюция становления LLM.

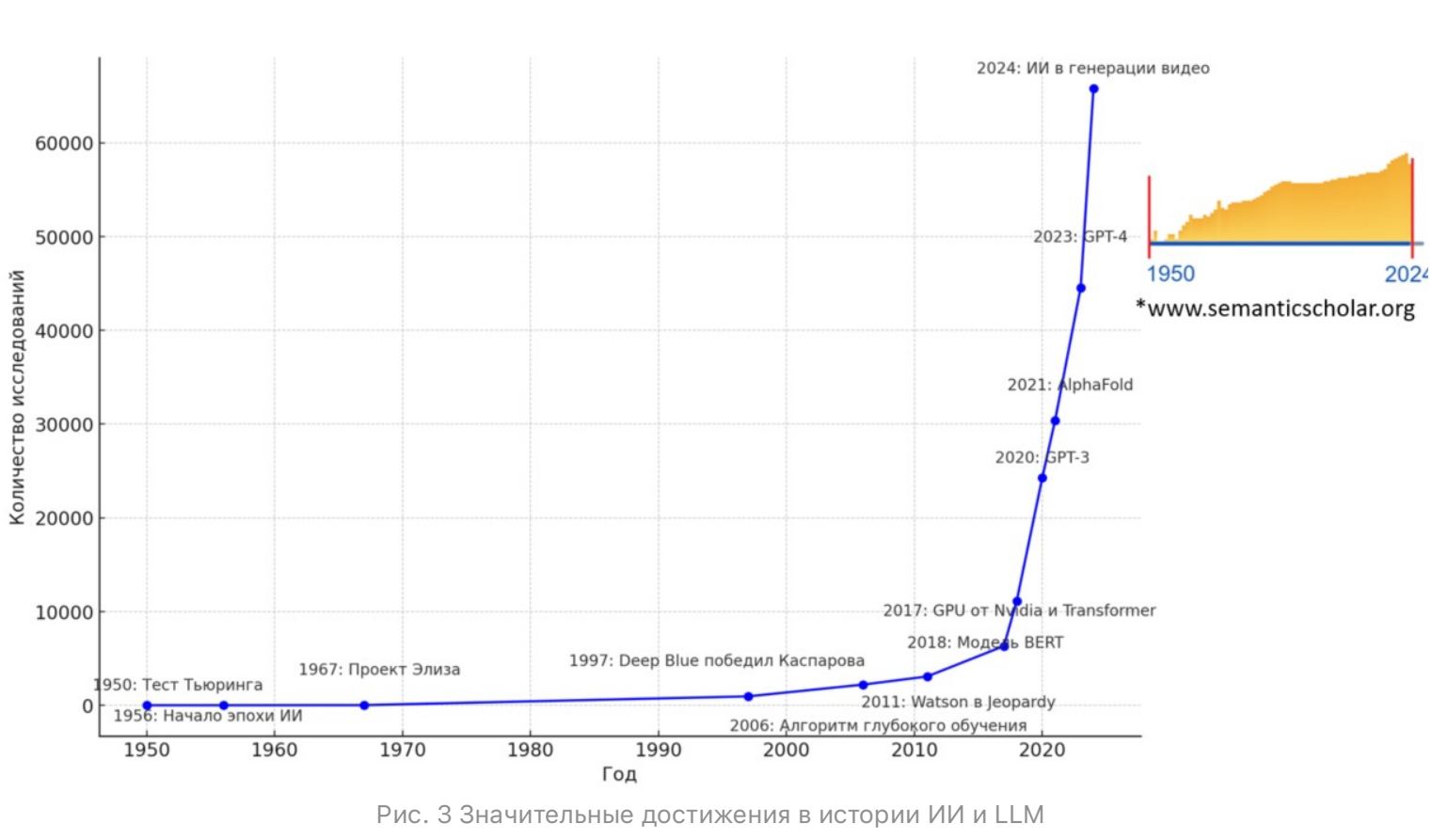


Рис. 3 Значительные достижения в истории ИИ и LLM

На небольшом графике в правом верхнем углу на рисунке 3 видно, что количество исследований в области искусственного интеллекта не всегда линейно росло. Эти исследования переживали периоды скачкообразного развития от одного достижения к другому, в то время как сейчас развитие экспоненциальное.

Что под капотом?

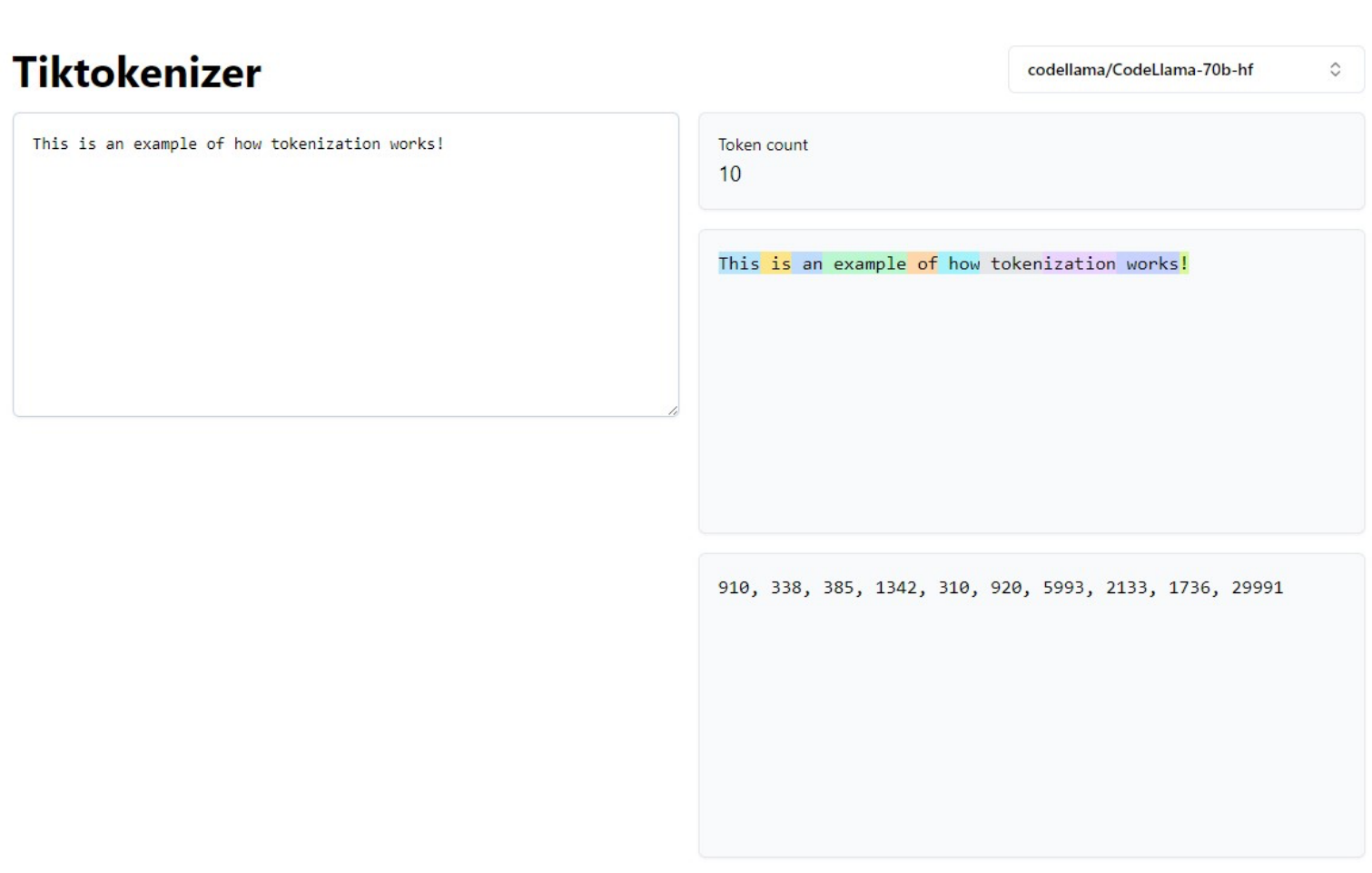
1.1 Основные понятия и параметры LLM

Промпт (запрос) — это вводимые пользователем данные, которые модель использует для ответа.

Токены — дискретные символы, слова и другие фрагменты текста, которые используются для представления данных в структурированном формате.

Токенизация — это процесс преобразования данных в отдельные единицы, называемые токенами (рис. 4, рис. 5).

Как показано на рисунках 4 и 5, токеном может быть как одно слово, так и один символ, а количество токенов в фразе на русском языке используется гораздо большее, чем на английском.


Tiktokenizer
codellama/CodeLlama-70b-hf

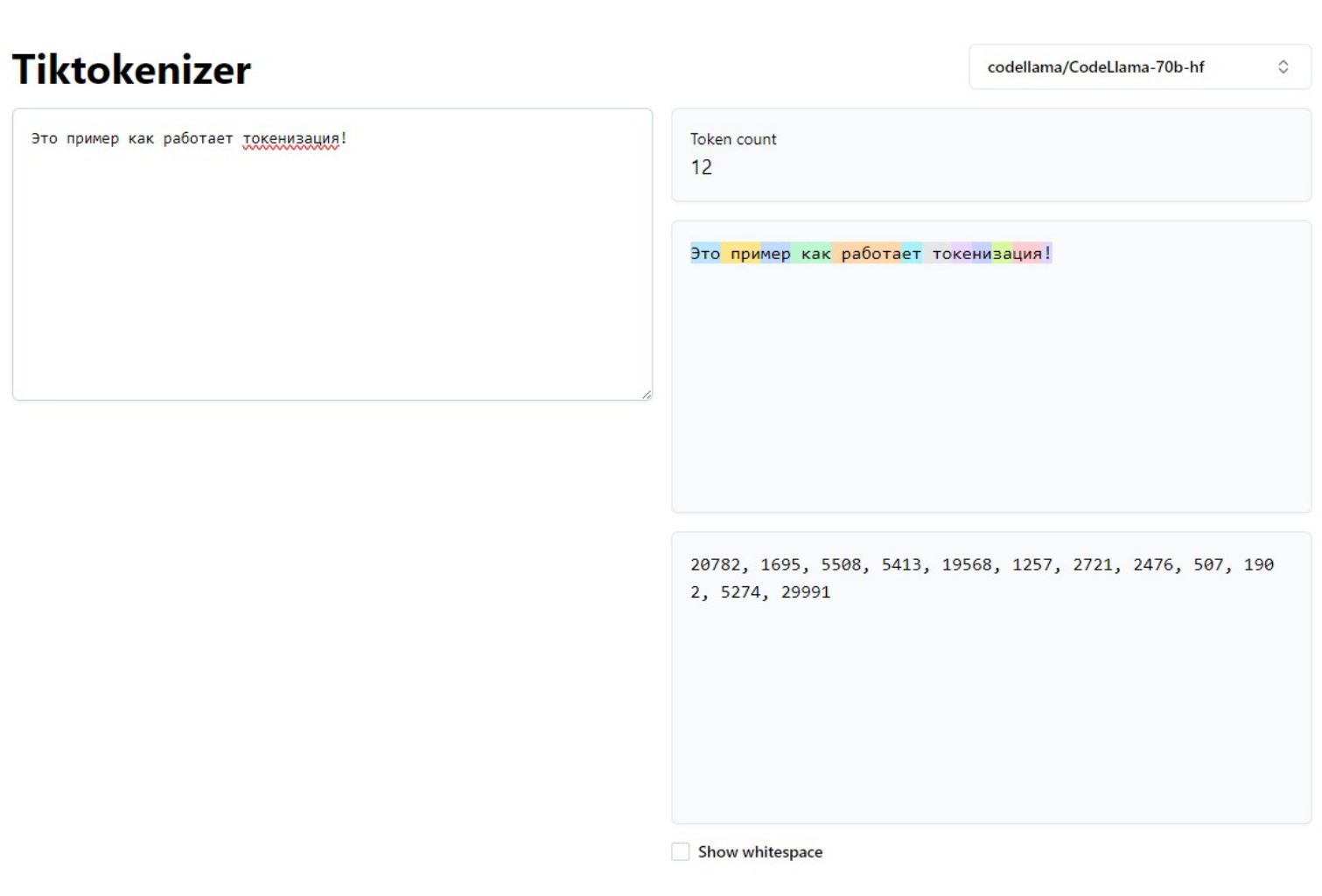
```
This is an example of how tokenization works!
```

Token count
10

```
This is an example of how tokenization works!
```

```
910, 338, 385, 1342, 310, 920, 5993, 2133, 1736, 29991
```

Рис. 4 Токенизация на английском


Tiktokenizer
codellama/CodeLlama-70b-hf
Show whitespace

```
Это пример как работает токенизация!
```

Token count
12

```
Это пример как работает токенизация!
```

```
20782, 1695, 5508, 5413, 19568, 1257, 2721, 2476, 507, 190
2, 5274, 29991
```

Рис. 5 Токенизация на русском

Контекстное окно — количество токенов, которые можно передать модели за раз (эквивалентно RAM в памяти компьютера).

Эмбеддинги — кодирование текста в смысл в виде вектора длины для базовой модели (сам вектор представляет собой смысл); или иначе — векторное представление слов в виде набора токенов.

Embedding model (модель встраивания) — тип LLM, который преобразует данные в векторы (массивы или группы чисел).

Базовая модель — это нейросеть, обученная на большом объёме данных, которую можно настроить для решения каких-либо задач.

Температура — параметр от 0 до 1, который влияет на креативность модели. При температуре близкой к 0 модель стремится дать более точный результат, при близкой к 1 выводит слова, которые менее часто встречались в обучающей выборке.

Top-p — выбор токенов из вариантов с наибольшей вероятностью; сумма их вероятностей определяет выбор модели.

Например, если p установлено равным 0,15, модель выберет такие токены, как «Юнайтед» и «Нидерланды», поскольку их вероятности составляют в сумме 14,7%, меньше 0,15, а «Чехию» уже проигнорирует (рис. 6). Чем ниже значение p, тем более стандартными являются ответы, генерируемые моделью. Общая рекомендация — изменить либо температуру, либо top-p, но не то и другое одновременно.

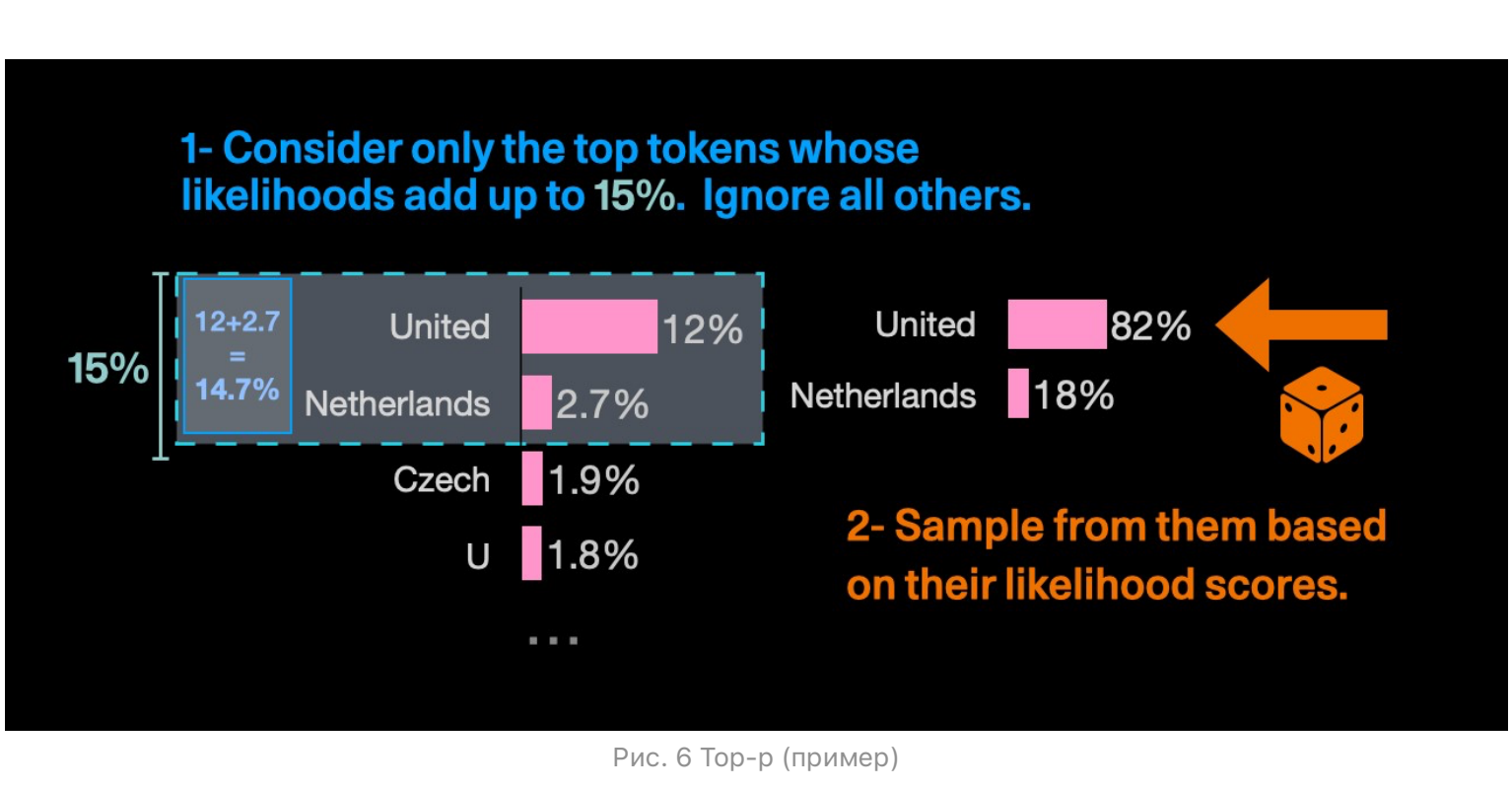

1- Consider only the top tokens whose likelihoods add up to 15%. Ignore all others.
15%
12+2.7 = 14.7%
United 12%
Netherlands 2.7%
Czech 1.9%
U 1.8%
...
United 82%
Netherlands 18%
2- Sample from them based on their likelihood scores.

Рис. 6 Top-p (пример)

Top-k — выбор следующего токена из списка токенов с наибольшим k, которые отсортированы по их вероятности (рис. 7). Например, если для k установлено значение 3, модель в соответствии с температурой выберет один из трёх лучших вариантов.

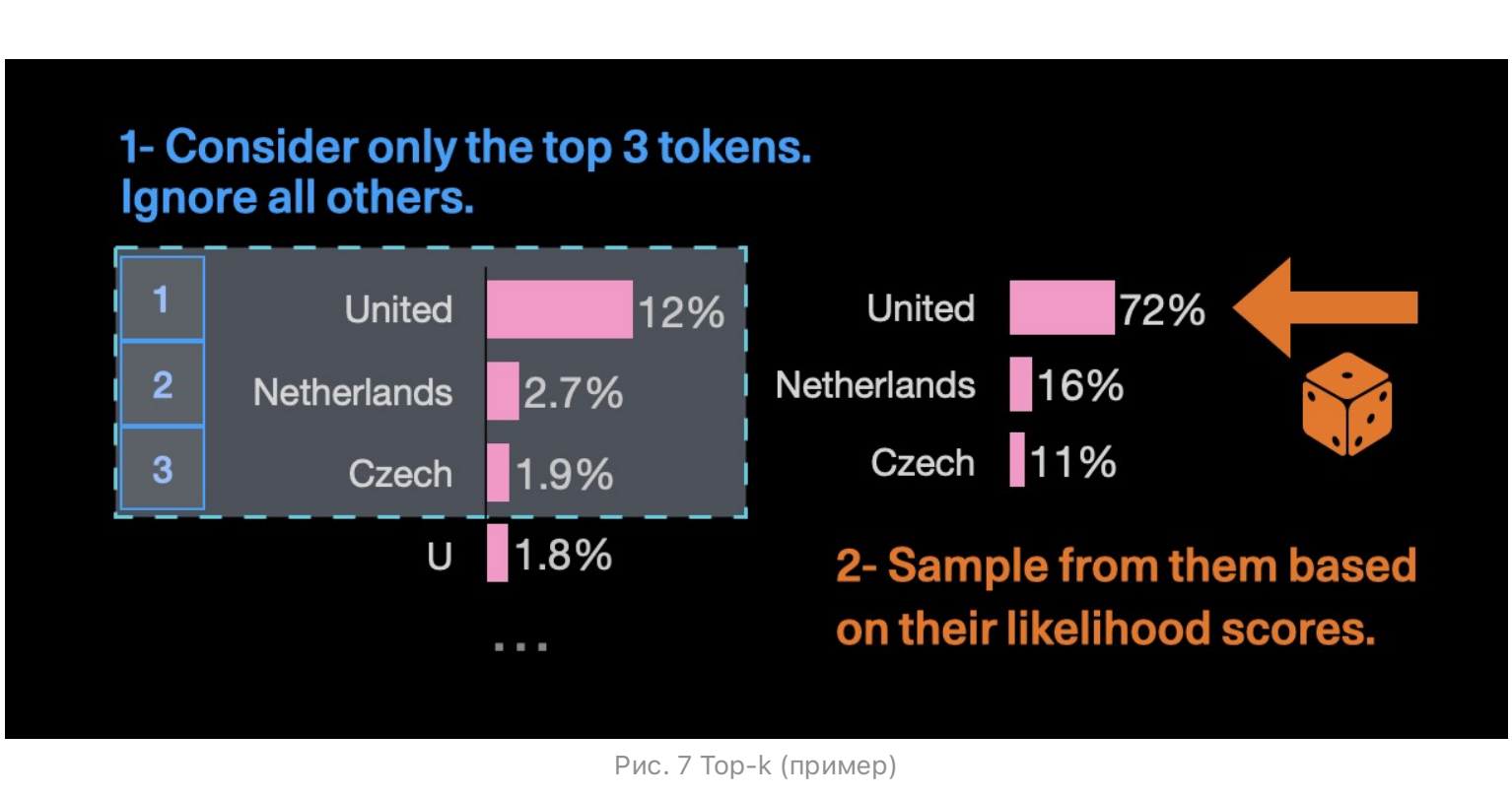

1- Consider only the top 3 tokens. Ignore all others.
1
2
3
United 12%
Netherlands 2.7%
Czech 1.9%
U 1.8%
...
United 72%
Netherlands 16%
Czech 11%
2- Sample from them based on their likelihood scores.

Рис. 7 Top-k (пример)

1.2 Состав LLM

Первые современные большие языковые модели с **2017 года** строятся на архитектуре Transformer, которая остаётся актуальной и в наши дни. **Трансформер** (Transformer) — базовая архитектура для многих современных моделей обработки естественного языка. На базе трансформера были созданы все имеющиеся большие языковые модели.

Применение архитектуры Transformer может варьироваться: например, может использоваться только декодер, только кодировщик или изменяться механизм работы слоя внимания и другие элементы. Однако несомненно то, что данная архитектура служит основой для всех существующих моделей в той или иной степени.

1.2.1 Архитектура Transformer и принцип работы

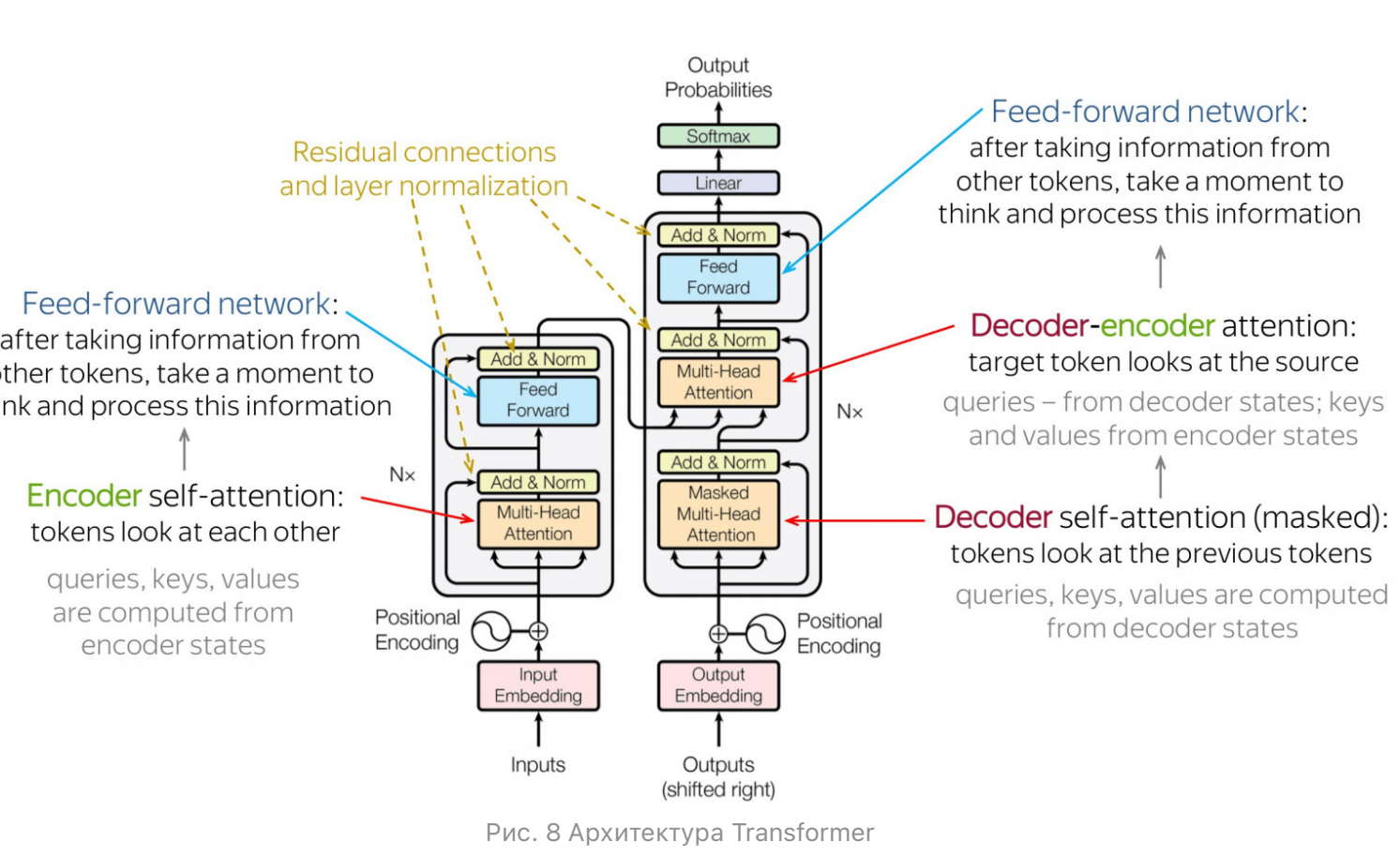

Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
Residual connections and layer normalization
Feed-forward network: after taking information from other tokens, take a moment to think and process this information
Encoder self-attention: tokens look at each other
queries, keys, values are computed from encoder states
Feed-forward network: after taking information from other tokens, take a moment to think and process this information
Decoder-encoder attention: target token looks at the source
queries – from decoder states; keys and values from encoder states
Decoder self-attention (masked): tokens look at the previous tokens
queries, keys, values are computed from decoder states

Рис. 8 Архитектура Transformer

Архитектура Transformer представляет собой мощную и гибкую модель для обработки последовательностей данных. Она состоит из двух основных частей: энкодера и декодера.

Энкодер (кодировщик) состоит из нескольких слоев, каждый из которых включает в себя два основных компонента (**Multi-Head Attention и Feed-forward network**).

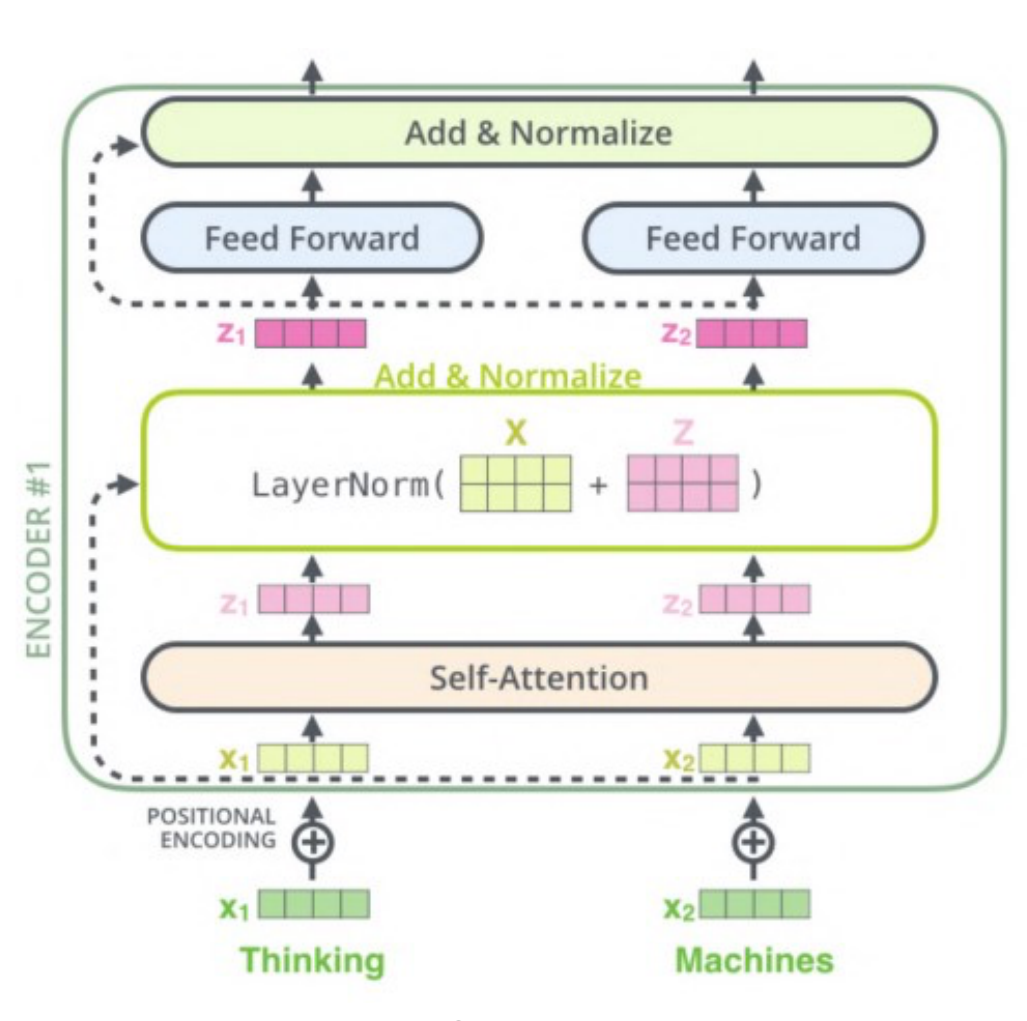

ENCODER #1
Add & Normalize
Feed Forward
Feed Forward
z1
z2
Add & Normalize
LayerNorm(X + Z)
z1
z2
Self-Attention
x1
x2
POSITIONAL ENCODING
x1
x2
Thinking
Machines

Рис. 9 Кодировщик

1. Multi-Head Attention | Self-attention (самовнимание):

- На вход поступают эмбедденги фиксированной величины, они перемножаются на веса (матрицы) — запросы, ключи и значения (queries, keys, values), затем конкатенируются и передаются обратно через линейный слой (Add & Norm), что улучшает способность модели устанавливать зависимости между токенами. Этот механизм разбивается на несколько «голов» (heads), каждая из которых использует свои собственные параметры внимания (Query, Key, Value).

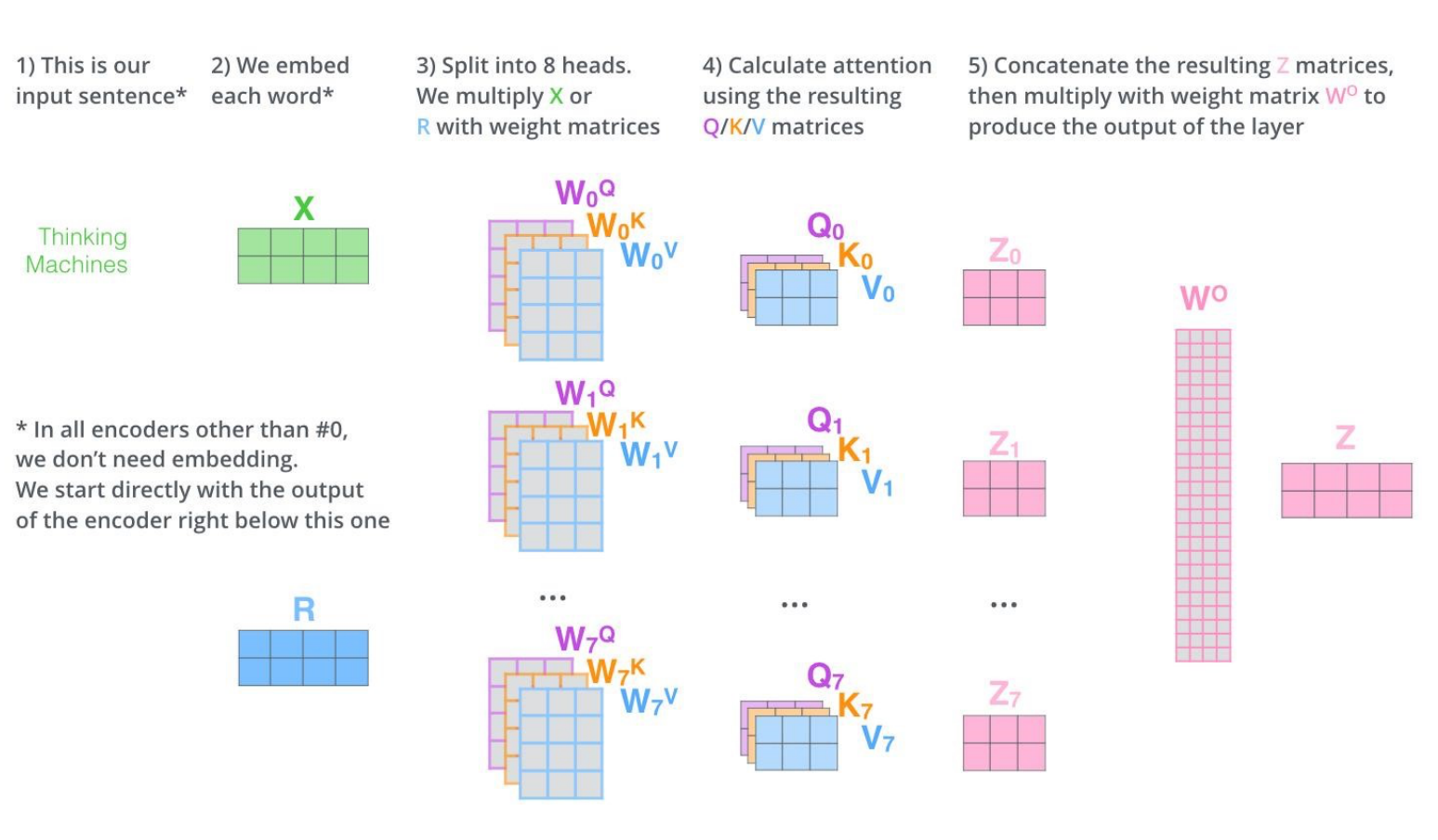

1) This is our input sentence*
2) We embed each word*
3) Split into 8 heads. We multiply X or R with weight matrices
4) Calculate attention using the resulting Q/K/V matrices
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer
Thinking Machines
X
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one
R
W0Q W0K W0V
W1Q W1K W1V
W7Q W7K W7V
Q0 K0 V0
Q1 K1 V1
Q7 K7 V7
Z0
Z1
Z7
WO
Z

Рис. 10 Multi-Head Attention

2. Feed-forward Neural Network (сеть прямого распространения):

- FNN представляет собой два обычных полносвязных слоя, применяемых независимо к каждому элементу входной последовательности. Размер промежуточного представления (выхода первого слоя) обычно в 4 раза больше выходов блока self-attention D_m >> D_ff.
- Между слоями используется нелинейная функция активации. Увеличенный размер промежуточного представления позволяет FNN обеспечить дополнительную нелинейность и экспрессивность модели. Два полносвязных слоя FNN с большим количеством параметров могут обрабатывать более сложные нелинейные зависимости в данных, а также позволяют более детально использовать веса, полученные с первого слоя, чтобы их применять к второму, который хранит в себе от 2/3 до 3/4 параметров модели Transformer.

Рис. 11 Feed-forward network

Декодер (декодировщик) также состоит из нескольких слоев, и каждый слой включает три основных компонента.

Рис. 12 Декодировщик

1. Masked Multi-Head Attention | self-attention (самовнимание):

Токены смотрят на предыдущие последовательности, чтобы учитывать контекст. Это достигается с помощью маскирования (треугольная матрица, где фиолетовые значения — нули), которое предотвращает появление информации о будущих токенах. Также на рисунке ниже представлено сравнение Self-Attention и masked Self-Attention.

Рис. 13 Masked Self-Attention

2. Encoder-decoder attention | Cross attention (внимание декодера к энкодеру):

- Декодер смотрит на состояния энкодера, чтобы учитывать информацию из исходной последовательности.
- Запросы берутся из состояний декодера, а ключи и значения из состояний энкодера.

Рис. 14 Encoder-decoder attention

3. Feed-forward Neural Network (сеть прямого распространения):

Аналогично энкодеру, после обработки внимания сеть обрабатывает и трансформирует информацию для её дальнейшей передачи следующему блоку.

Дополнительные элементы:

Каждый слой энкодера и декодера включает также пропускные соединения или остаточные связи (**Add & Norm**), которые позволяют улучшить обучение сети и стабилизировать градиенты:

- **Добавление (Add)**: выходные данные слоев внимания или FNN добавляются к исходным данным.
- **Нормализация (Norm)**: затем берётся среднее значение и стандартное отклонение результатов, которые вычисляются для каждой обучающей выборки.

Positional encoding (позиционное кодирование): поскольку трансформеры не имеют встроенного порядка следования токенов, позиционное кодирование добавляет информацию о позиции каждого токена в последовательности.

Выходной слой:

Linear layer (линейный слой) преобразует выходные данные декодера в логиты (сырые значения), которые будут использоваться для предсказания вероятностей следующего токена.

Softmax преобразует логиты в вероятности, которые используются для выбора следующего токена в последовательности.

Принцип работы (кратко)

1. Входная последовательность проходит через слой входного кодирования, добавляются позиционные кодировки.
2. Энкодер обрабатывает последовательность через несколько слоев самовнимания и прямого распространения.
3. Декодер принимает сдвинутую вправо выходную последовательность, проходит через слои маскированного самовнимания, внимания к энкодеру и прямого распространения.
4. Линейный слой и Softmax преобразуют выходные данные декодера в вероятности предсказания следующего токена.

Классификация моделей

LLM могут иметь различную архитектуру в зависимости от их целей, вычислительных ресурсов и задач обучения. Также их можно разделить по архитектурам:

- **(На базе энкодера) BERT** (Bidirectional Encoder Representations from Transformers) — модель, которая смотрит на текст в обоих направлениях для лучшего понимания контекста.
- **(На базе декодера) GPT** (Generative Pre-trained Transformer) — модель, предназначенная в первую очередь для генерации текста.
- **(Энкодер + декодер) T5** (Text-to-Text Transfer Transformer) — модель, которая рассматривает все задачи обработки естественного языка как преобразование текста в текст.
- **MoE** (Mixture of Experts) — модель, которая состоит из нескольких экспертов (предобученных моделей) для выполнения определённой задачи.

В таблице 1.1 приведено сравнение этих моделей в более удобном формате.

Таблица 1.1 Сравнение архитектур больших языковых моделей:

Архитектура	Применение	Пример модели
Только энкодер	Классификация, извлечение признаков	BERT, RoBERTa
Только декодер	Генерация текста	GPT, Llama
Энкодер и декодер	Машинный перевод, пересказ текста	T5, BART
Несколько экспертов	В зависимости от имеющихся моделей	Mistral

Немного практики

Существует множество реализаций примеров по использованию API OpenAI, но если есть необходимость проводить локальные эксперименты для создания своих собственных приложений и качество не очень важно, то можно рассмотреть использование готовых open-source-моделей.

На сайте существует большое количество различных моделей с разнообразной классификацией:

- по языку,
- по типу задачи,
- по датасету и т. д.

Рис. 15 Hugging Face. Главная

Примеры использования моделей бывают сразу представлены в проектах автора. В качестве примера возьмём проект Ильи Гусева «mbart_ru_sum_gazeta» для краткого пересказа:

Рис. 16 Hugging Face, экземпляр кода

Немного переработав код, получилось создать примитивное локальное web-приложение для пересказа текста. В связи с тем, что данные для дообучения были созданы на основе новостных изданий, то текст для демонстрации был выбран из новостного источника.

Рис. 17 WEB-интерфейс для модели mbart

Полезные источники

Где можно попробовать топовые модели бесплатно:

- Конкурент ChatGPT с открытым исходным кодом (появился во время написания статьи);
- Арена для сравнения моделей;
- Слепое тестирование. После голосования отображаются используемые модели.

Локальные эксперименты:

- Ollama + tutorial;
- LM Studio - Discover, download, and run local LLMs — очень простое приложение для использования языковых моделей локально.

Также:

- Сайт, на котором собирается крайне большое количество LLM-инструментов или агентов, предназначенных для различных задач. Есть как платные, так и бесплатные инструменты.
- Сайт, который поможет написать статью.
- Поиск научной литературы, отображение цитирования литературы в виде графа.
- Инструмент, позволяющий в табличном формате сравнить и представить актуальную литературу.

Георгий Абраменко
Инженер

Теги: искусственный интеллект, нлп, нейросети, машинное обучение, llm, large language model, большая языковая модель

Хабы: Блог компании Газинформсервис, Искусственный интеллект, Natural Language Processing, Машинное обучение

Редакторский дайджест
Присылаем лучшие статьи раз в месяц
Электропочта
Подписаться
Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

4K+
Охват за 30 дней
Газинформсервис
Сайт Telegram ВКонтакте

3
Карма
@TheSeCure
Пользователь
Подписаться

Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

pilot_artem 14 часов назад
Что там с Ту-214
11 мин 24K
+72 23 123 +123

shiru8bit 20 часов назад
Трекеры и чиптюн на AdLib
Простой 22 мин 12K
Ретроспектива
+33 7 11 +11

monobogdan 19 часов назад
Sony CMD-J7: шедевр инженерной мысли из 2001'ого
Простой 6 мин 15K
Ретроспектива
+30 16 15 +15

gliderman 17 часов назад
Как за 300 рублей и 6 часов получить прошивку уровня синьора?
Простой 4 мин 22K
Кейс
+23 67 39 +39

RationalAnswer 4 часа назад
Налоговая слежка за переводами на карту, а также начало суда Маска против Альтмана
7 мин 9.2K
Дайджест
+19 5 4 +4

Ungated 16 часов назад
Как устроен Meshtastic, зачем он нужен и как я подключил его к локальной модели на ноутбуке
Средний 21 мин 17K
Кейс
+17 81 18 +18

CatScience 19 часов назад
Обрекли на учебники: как праиндоевропейцы развалили свой язык
3 мин 14K
+17 5 21 +21

TrexSelectel 1 час назад
Как гигантским корпорациям стало выгоднее вкладываться в AI, чем в майнинг биткоина
9 мин 2.9K
+16 1 1 +1

Catx2 21 час назад
КлокВоркПанк
4 мин 12K
+16 14 3 +3

AGmind 23 часа назад
Как я собрал на DGX Spark приватный AI-сервер, и теперь рассказываю, что туда вошло
Средний 15 мин 12K
Обзор Из песочницы
+16 46 26 +26

Что нам стоит IaaS построить: как мы собирали прочный фундамент для облака
Турбо

Показать еще

ВАКАНСИИ КОМПАНИИ «ГАЗИНФОРМСЕРВИС»

Специалист по тестированию систем искусственного интеллекта
ГАЗИНФОРМСЕРВИС · Санкт-Петербург · Можно удаленно

Специалист по инфраструктуре машинного обучения (MLOps-инженер)
ГАЗИНФОРМСЕРВИС · Санкт-Петербург

Менеджер направления СОК (SOC GSOC)
ГАЗИНФОРМСЕРВИС · Санкт-Петербург

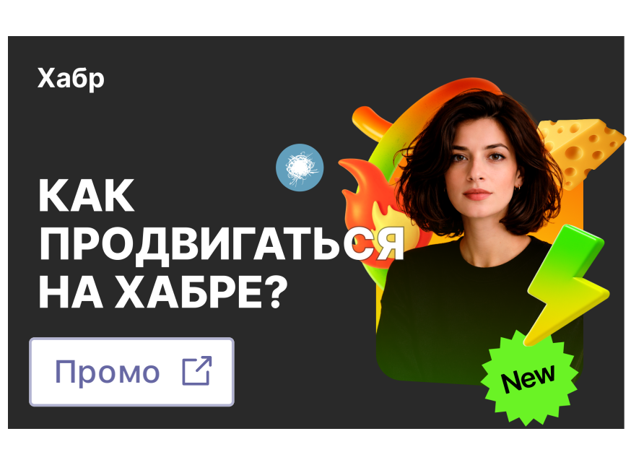
Инженер внедрения систем мониторинга ИБ
ГАЗИНФОРМСЕРВИС · Санкт-Петербург

Инженер по анализу защищенности инфраструктуры (тестирование на проникновение)
ГАЗИНФОРМСЕРВИС · Санкт-Петербург

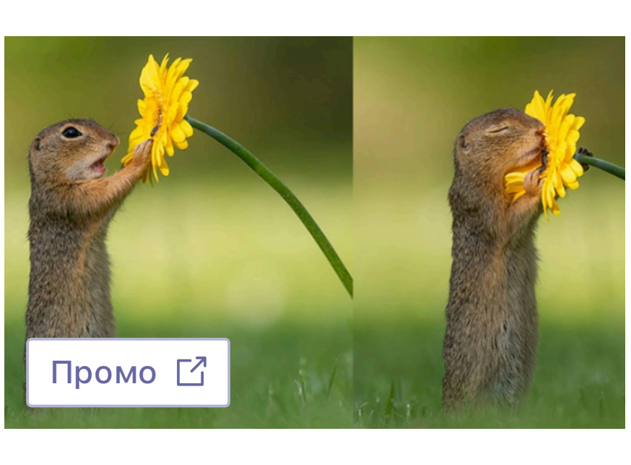
Больше вакансий на Хабр Карьере

МИНУТОЧКУ ВНИМАНИЯ

Турбо
Удалёнка в фарме: больше данные и никакой магии



Промо
Рассказываем в обновленном и расширенном курсе



Промо
Запахло весной и скидками в Промокодусе

Хабр

Настройка языка

Техническая поддержка

© 2006–2026, Habr